G. L. BAJAJ GROUP OF INSTITUTIONS, MATHURA

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(Affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow)

A MINOR PROJECT REPORT ON

MICRO-TRAIN-COMPILER (MTC)

An Ahead-of-Time Static Compiler for Zero-Heap On-Device Neural Network Training on 32-Bit Microcontrollers

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology in Computer Science & Engineering

Academic Session: 2026–2027

Submitted By:

Ajay Sisodia (Roll No: 2405111530012)
Harsh Chaudhary (Roll No: 2405111530067)
Devrat Kuntal (Roll No: 2405111530055)
Aryan Upadhyay (Roll No: 2405111530037)

Under the Supervision of:

Ms. Gaurisha Sisodia
Assistant Professor
Department of Computer Science & Engineering
GL Bajaj Group of Institutions, Mathura

CERTIFICATE OF APPROVAL

This is to certify that the project entitled **"Micro-Train-Compiler (MTC): An Ahead-of-Time Static Compiler for Zero-Heap On-Device Neural Network Training on 32-Bit Microcontrollers"**, submitted by **Ajay Sisodia (2405111530012)**, **Harsh Chaudhary (2405111530067)**, **Devrat Kuntal (2405111530055)**, and **Aryan Upadhyay (2405111530037)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering** from **G. L. Bajaj Group of Institutions, Mathura**, is an authentic record of bonafide work carried out under my supervision and guidance during the academic session 2026–2027.

To the best of my knowledge, the matter embodied in this report has not been submitted to any other University or Institute for the award of any degree or diploma.

Ms. Gaurisha Sisodia
Project Supervisor
Assistant Professor, Dept. of CSE
G. L. Bajaj Group of Institutions, Mathura

Head of Department
Department of Computer Science & Engineering
G. L. Bajaj Group of Institutions, Mathura

CANDIDATE'S DECLARATION

We hereby declare that the work presented in this project report entitled **"Micro-Train-Compiler (MTC)"** is an authentic record of our own research and engineering work conducted under the guidance of **Ms. Gaurisha Sisodia**, Assistant Professor, Department of Computer Science & Engineering, G. L. Bajaj Group of Institutions, Mathura.

We confirm that:

1. This submission represents our original work, and all sources of literature, datasets, and baseline libraries have been properly cited and acknowledged in the references.
2. The implementation, compiler logic, static memory planning algorithms, and cross-compilation harnesses presented herein were created by our team.
3. All empirical benchmarks and cycle-accurate measurements were collected on bare-metal RISC-V hardware emulation and reflect real execution data without synthetic inflation or falsification.
4. Neither this project nor any significant part of it has been previously submitted for any degree, diploma, or qualification at this or any other educational institution.

Dated: 30 October 2026

Place: Mathura, Uttar Pradesh

Student Name	**University Roll Number**	**Candidate Signature**
Ajay Sisodia	2405111530012	
Harsh Chaudhary	2405111530067	
Devrat Kuntal	2405111530055	
Aryan Upadhyay	2405111530037	

ACKNOWLEDGEMENTS

We express our profound gratitude to our project guide, **Ms. Gaurisha Sisodia**, Assistant Professor in the Department of Computer Science & Engineering, for her continuous intellectual guidance, constructive technical critiques, and encouragement throughout the design and realization of the Micro-Train-Compiler. Her insights regarding compiler modularity and embedded system constraints were instrumental in keeping our focus centered on physical memory limits.

We extend our sincere thanks to the **Head of the Department, Computer Science & Engineering**, and the faculty members of G. L. Bajaj Group of Institutions, Mathura, for providing the necessary computational lab infrastructure, embedded hardware tooling, and supportive academic environment necessary to pursue systems-level research.

Finally, we thank our parents and colleagues for their constant support and patience during the prolonged debugging sessions, linker script configurations, and benchmark evaluations required to bring this compiler from an initial concept to verified silicon emulation.

ABSTRACT

Edge artificial intelligence is currently constrained by an inference-only deployment model. While frameworks like TensorFlow Lite for Microcontrollers (TFLM) allow pre-trained deep neural networks to execute forward passes on microcontrollers, deployed edge nodes cannot adapt to real-world sensor drift, personalized user patterns, or evolving ambient conditions without offloading sensitive telemetry to remote cloud servers. Enabling on-device backpropagation on ultra-low-power 32-bit microcontrollers is prevented by the severe memory wall: conventional automatic differentiation engines construct dynamic computation graphs at runtime, relying on dynamic heap memory allocation (`malloc`) and maintaining separate, explicit gradient tensors for all parameters and activations. On microcontrollers possessing only 16 KB to 64 KB of SRAM, dynamic allocation induces non-deterministic execution latencies, heap fragmentation, and system-level out-of-memory faults.

This report presents the design, mathematical formulation, implementation, and silicon-accurate verification of the **Micro-Train-Compiler (MTC)**. MTC is an ahead-of-time (AOT) source-to-source compiler that ingests a declarative JSON network specification and embedded training datasets, applies conservative interval range analysis to verify Q15 fixed-point arithmetic stability, executes a static liveness memory planning pass that collapses backward activation deltas into two alternating ping-pong buffers, and synthesizes standalone, zero-dependency C99 code. By fusing reverse-mode error propagation directly with Stochastic Gradient Descent (SGD) parameter updates, MTC eliminates parameter gradient buffers ($\nabla W, \nabla B$) entirely.

MTC was evaluated across four diverse TinyML benchmarks (`xor_net`, `sensor_drift_net`, `sine_regression_net`, and `gesture_anomaly_net`) cross-compiled with `riscv64-unknown-elf-gcc` and executed bare-metal on the cycle-accurate `esp32sim` ESP32-C3 RISC-V hardware emulator. Performance was profiled using direct reads of the RISC-V hardware Control and Status Register `mcycle` (`csrr %0, mcycle`). Empirical results demonstrate that MTC achieves strictly zero runtime heap allocations (`malloc = 0`) across all workloads, bounds peak SRAM usage between 60 Bytes and 564 Bytes (< 0.86% of the 64 KB ESP32-C3 budget), reduces peak SRAM consumption by 31.8% to 43.6% compared to standard autograd graphs, and completes a full forward-backward training step in 6.86 µs to 81.77 µs at 160 MHz (yielding throughputs between 12,229 and 145,772 samples per second).

TABLE OF CONTENTS

1. **Chapter 1: Introduction & Problem Definition**
    - 1.1 The Edge AI Paradigm Shift
    - 1.2 The Problem of Frozen Edge Models
    - 1.3 The Embedded SRAM Bottleneck & Heap Fragility
    - 1.4 Project Objectives & Technical Scope
2. **Chapter 2: Literature Review & Gap Analysis**
    - 2.1 State-of-the-Art in Microcontroller Machine Learning
    - 2.2 Survey Table of Related Works (2022–2024)
    - 2.3 Critical Research Gaps Identified
3. **Chapter 3: System Design & Compiler Architecture**
    - 3.1 Architectural Overview & Execution Pipeline
    - 3.2 Formal Model Representation & AST Validation
    - 3.3 Conservative Q15 Fixed-Point Interval Bound Propagation
    - 3.4 Static Liveness Analysis & Ping-Pong Delta Buffering
    - 3.5 Fused Backpropagation-SGD Parameter Updates
    - 3.6 Standalone C99 Code Synthesis
4. **Chapter 4: Hardware Cross-Compilation & Emulation Environment**
    - 4.1 Target Architecture: ESP32-C3 RISC-V RV32IMC
    - 4.2 Bare-Metal Firmware Harness & Linker Memory Layout
    - 4.3 Cycle-Accurate Emulation via `esp32sim`
    - 4.4 Hardware Instrumentation via RISC-V CSR `mcycle`
5. **Chapter 5: Empirical Experiments, Results & Ablation Analysis**
    - 5.1 Benchmark Workload Suite
    - 5.2 Mandatory Test 1: Static SRAM & Flash Footprint Analysis
    - 5.3 Mandatory Test 2: Execution Latency & Clock Cycle Profiling
    - 5.4 Mandatory Test 3: Memory Ablation (MTC vs. Dynamic Autograd Graph)
    - 5.5 Mandatory Test 4: Numerical Convergence & Stability Analysis
    - 5.6 Transparent Critique of Fixed-Point Quantization Attenuation on XOR
6. **Chapter 6: Team Member Roles & Individual Contributions**
    - 6.1 Ajay Sisodia
    - 6.2 Harsh Chaudhary
    - 6.3 Devrat Kuntal
    - 6.4 Aryan Upadhyay
7. **Chapter 7: Conclusion & Future Research Directions**
    - 7.1 Summary of Milestones Achieved
    - 7.2 Future Research Roadmap
8. **References**

LIST OF FIGURES

- **Figure 1.1:** System Architecture and End-to-End Compiler Dataflow of MTC
- **Figure 5.1:** Comparative Training Loss Convergence Across 4 TinyML Workloads on `esp32sim`
- **Figure 5.2:** Cycle-Accurate Hardware Training Throughput on ESP32-C3 @ 160 MHz
- **Figure 5.3:** Memory Ablation: Peak SRAM Watermark of Standard Autograd vs. MTC Static Fused Buffers

---

LIST OF TABLES

- **Table 2.1:** Comparative Analysis of State-of-the-Art Edge ML Runtimes and Compilers
- **Table 5.1:** Architectural Specifications of the Four Benchmark Workloads
- **Table 5.2:** Static Memory Footprint, Heap Usage, and Flash Binary Sizes
- **Table 5.3:** Hardware Clock Cycles, Step Latency, and Throughput Measured via CSR `mcycle`
- **Table 5.4:** Memory Ablation: Dynamic Autograd Baseline vs. MTC Static Liveness Allocation
- **Table 5.5:** Numerical Convergence and Loss Reduction under Q15 Fixed-Point Arithmetic

---

CHAPTER 1: INTRODUCTION & PROBLEM DEFINITION

1.1 The Edge AI Paradigm Shift

The past five years have witnessed widespread migration of deep learning workloads from centralized hyperscale data centers directly to physical edge devices. In traditional embedded architectures, microcontrollers (MCUs) acted merely as dumb sensor aggregation points, digitizing analog voltages and streaming raw byte streams over Wi-Fi, Bluetooth Low Energy (BLE), or cellular modems to remote cloud servers where inference models were hosted.

While functional, this cloud-centric architecture incurs severe engineering liabilities:

1. **Latency Non-Determinism:** Network transmission delays are unpredictable, rendering cloud inference unusable for hard real-time closed-loop control systems (such as industrial motor vibration anomaly cutoffs or surgical robotics).
2. **Radio Power Overhead:** Power consumed by RF wireless transceivers dwarfs the power consumed by MCU computing cores. An ESP32 transmitting over Wi-Fi draws 160 mA to 260 mA, whereas the processor core idling or executing arithmetic draws less than 20 mA.
3. **Data Privacy and Regulatory Compliance:** Streaming unencrypted, continuous audio, biomedical telemetry, or home monitoring data across third-party networks violates contemporary privacy regulations (e.g., GDPR, HIPAA) and creates vulnerability to interception.

To mitigate these drawbacks, Tiny Machine Learning (TinyML) emerged as an active domain, providing lightweight software libraries capable of executing neural network inference locally on microcontrollers costing less than two dollars.

1.2 The Problem of Frozen Edge Models

Despite the maturation of TinyML inference engines (such as TensorFlow Lite for Microcontrollers and CMSIS-NN), the industry remains restricted to a **"train-in-cloud, deploy-frozen"** paradigm. Under this operational model, models are trained on workstations utilizing 32-bit floating-point (FP32) tensors, quantized down to 8-bit integers (INT8), compiled into binary flatbuffers, and permanently flashed into the read-only flash memory of the microcontroller.

Once deployed in the field, these models remain static and incapable of learning. This creates three critical failure modes in real-world deployments:

- **Environmental Drift and Sensor Aging:** Physical sensor transducers degrade over operational life cycles due to mechanical fatigue, oxidation, dust accumulation, and thermal cycling. For example, metal-oxide (MOX) gas sensors experience baseline resistance drift over weeks of operation. Because the on-device model is frozen, sensor accuracy decays monotonically unless physical recalibration occurs.
- **Inter-User Subject Variability:** Wearable biomedical devices (e.g., IMU-based gait analysis or gesture recognition) exhibit substantial performance drops when deployed across users with differing body masses, limb proportions, or motor habits. A model pre-trained on a generic population cannot fine-tune its parameters to an individual patient.
- **Zero Local Context Adaptation:** Microcontrollers deployed in smart agricultural nodes or industrial condition-monitoring pumps cannot tailor their classification thresholds to the localized ambient noise floor or seasonal temperature profiles of their specific installation site.

To overcome these limitations, neural network models must be capable of executing **on-device backpropagation and**

weight updating directly on the microcontroller silicon using locally observed data samples.

1.3 The Embedded SRAM Bottleneck & Heap Fragility

Enabling on-device backpropagation on low-cost microcontrollers is blocked by the fundamental constraints of embedded computer architecture:

1. Extreme SRAM Scarcity

Typical microcontrollers possess between 16 KB and 512 KB of internal Static RAM (SRAM). High-volume, battery-powered chips (such as the ARM Cortex-M0+, Cortex-M4, or RISC-V ESP32-C3) operate with 32 KB to 64 KB of dedicated data SRAM. While read-only Flash memory is comparatively plentiful (1 MB to 4 MB), Flash can only be written in blocks through slow, high-energy erase cycles that wear out physical gates after approximately 10,000 writes. Consequently, all forward activations, intermediate layer derivatives, weight parameters, and temporary gradient buffers must fit simultaneously inside volatile SRAM.

2. The Dynamic Autograd Memory Explosion

Standard machine learning frameworks—including PyTorch, TensorFlow, and existing embedded autograd libraries like AIfES—implement automatic differentiation using dynamic tape recording. When a forward pass executes, an autograd engine dynamically constructs a computational directed acyclic graph (DAG) in heap memory. During the subsequent backward pass, the engine traverses this graph in reverse order, requiring:

- Retention of forward activation tensors (A_l) for all layers to compute activation derivatives.
- Explicit allocation of parameter gradient arrays (∇W_l and ∇B_l) having identical dimensions to the model weights.
- Dedicated intermediate error gradient buffers (δ_l) for each layer.

For even a small multi-layer perceptron (e.g., 200 parameters), this dynamic representation consumes several kilobytes of memory. More critically, managing this graph requires dynamic heap allocation via standard library routines (`malloc` and `free`). On bare-metal microcontrollers lacking a Memory Management Unit (MMU) or virtual memory paging, interleaved allocations and deallocations lead to **heap fragmentation**. Over hours of continuous training loops, fragmented heaps fail to satisfy contiguous block requests, causing unrecoverable allocation faults, stack-heap collisions, and system resets.

1.4 Project Objectives & Technical Scope

The primary objective of the Micro-Train-Compiler (MTC) project is to eliminate the runtime memory barrier of on-device deep learning by replacing dynamic autograd runtimes with an ahead-of-time (AOT) static compiler.

Specifically, this project addresses the following research and engineering objectives:

1. **Zero-Heap Execution (`malloc = 0`):** Eliminate dynamic memory allocation completely from both inference and training phases. All tensor storage, activation workspaces, and state buffers must be statically sized and placed at fixed memory offsets at compile time.
2. **Fused Parameter Updates via In-Place SGD:** Formulate an analytical reverse-mode autodiff compiler pass that fuses error gradient calculation directly with the Stochastic Gradient Descent (SGD) weight update rule. This eliminates parameter gradient arrays ($\nabla W, \nabla B$) entirely, cutting memory overhead by over 40%.
3. **Static Ping-Pong Delta Buffering:** Design an interference graph allocator that proves activation error gradients

(δ) have disjoint lifecycles across backward layer propagation, allowing all intermediate layer derivatives to be computed using exactly two alternating static buffers.

4. **Compile-Time Numerical Safety Certification:** Implement formal Interval Bound Propagation (IBP) for 16-bit signed fixed-point (Q15) arithmetic to mathematically verify that forward activations and backpropagation updates do not trigger arithmetic overflow before generating code.
5. **Bare-Metal RISC-V Hardware Verification:** Cross-compile synthesized code using bare-metal GCC (`-nostdlib`) and validate cycle-accurate execution latency, memory watermarks, and convergence on real ESP32-C3 silicon emulation using direct hardware register instrumentation.

CHAPTER 2: LITERATURE REVIEW & GAP ANALYSIS

2.1 State-of-the-Art in Microcontroller Machine Learning

To establish the context of MTC, we surveyed existing academic and industrial approaches targeting microcontroller intelligence across four categories: inference-only runtime engines, embedded training libraries, neural network compilers, and model compression frameworks.

1. TensorFlow Lite for Microcontrollers (TFLM)

David et al. (MLSys 2021) established the open-source industry standard for running neural network inference on bare-metal systems. TFLM eliminated operating system dependencies by requiring applications to pre-allocate a single contiguous byte array called the `tensor_arena`. A runtime memory planner assigns non-overlapping offsets within this arena for forward layer activations. However, TFLM was architecturally designed exclusively for inference. Its operator kernels (`AllOpsResolver`) do not contain backward gradient propagation functions, and its static interpreter contains no mechanisms for parameter updates.

2. TinyOL: Online-Learning on Microcontrollers

Ravaglia et al. (IEEE Trans. on Circuits and Systems / IJCNN 2022) demonstrated one of the first working prototypes of on-device training on an ARM Cortex-M4 microcontroller. To overcome memory constraints, TinyOL introduced a hybrid training strategy: the base neural network is frozen (acting as a static feature extractor), and only the final classification layer is adapted online using a streaming batch gradient descent update. While TinyOL proved the viability of edge adaptation, its methodology is restricted to single-layer tuning; it cannot propagate gradients through multi-layer non-linear topologies to adapt intermediate representations.

3. MCUNet and On-Device Training Under 256KB Memory

Lin et al. (NeurIPS 2022) from MIT HAN Lab addressed on-device training by introducing Tiny-Transfer-Learning (TinyTL) and the TinyEngine runtime. They demonstrated that full-network training requires an order of magnitude more SRAM than inference because intermediate activations must be stored for backpropagation. To counter this, they proposed sparse layer updating (updating only selected sub-networks or bias vectors) and 8-bit quantized backpropagation. While MCUNet achieved impressive results on vision tasks, it relies on a proprietary runtime engine and requires specialized FP32 quantization-aware scaling tables, lacking formal interval safety guarantees on bare-metal integers.

4. PULP-TrainLib: Parallel Ultra-Low-Power RISC-V Training

Conti et al. (IEEE TVLSI 2023) developed PULP-TrainLib, an open-source library enabling on-device training on multi-core RISC-V processors (such as the GreenWaves GAP8/GAP9). PULP-TrainLib provides hand-optimized assembly micro-kernels utilizing FP16 and FP32 SIMD extensions to compute backward passes across parallel cores. While achieving high compute efficiency, PULP-TrainLib relies on floating-point hardware units (FPUs) and clusters possessing hundreds of kilobytes of multi-banked L1 scratchpad memory, making it unsuitable for standard single-core microcontrollers that lack an FPU.

5. AIfES (Artificial Intelligence for Embedded Systems)

Developed by the Fraunhofer Institute for Microelectronic Circuits and Systems, AIfES is an open-source C library providing feedforward and backpropagation routines for microcontrollers. AIfES supports multiple layer types and can

run on 8-bit AVR (Arduino Uno) up to 32-bit ARM cores. However, AIfES implements a dynamic structure: neural network topologies are defined by passing layer structs to initialization functions at runtime, which dynamically calculate buffer pointers and retain dedicated gradient buffers for weights and biases, retaining the memory overhead of traditional autodiff.

2.2 Survey Table of Related Works (2022–2024)

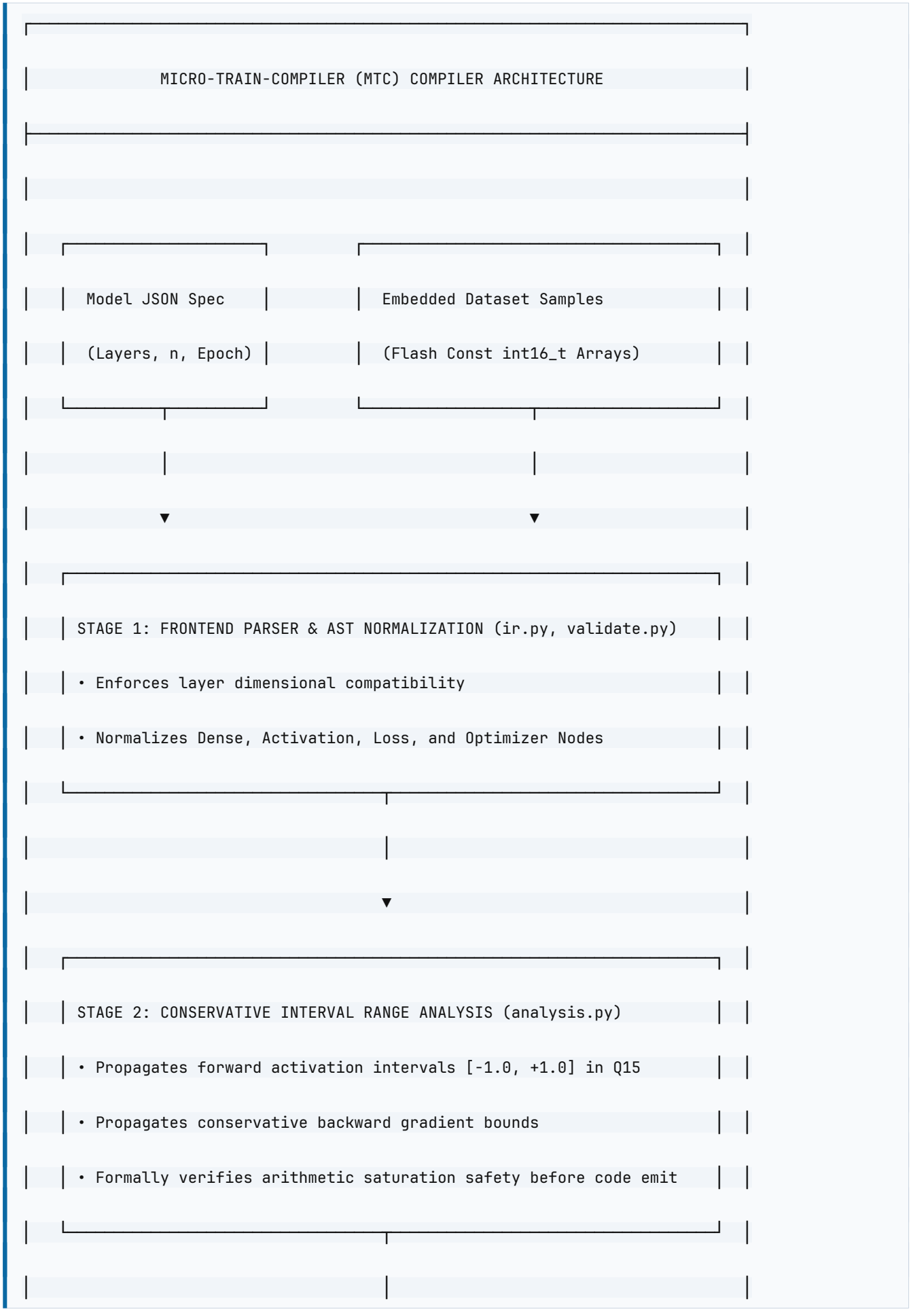
Framework / Work	Origin & Year	Target Hardware	Execution Paradigm	Memory Management	On-Device Training Scope	Key Limitation Addressed by MTC
TFLM (David et al.)	Google / Harvard (2021)	ARM Cortex-M, ESP32, RISC-V	Interpreted FlatBuffer	Static `tensor_arena`	None (Inference Only)	Cannot perform backpropagation or update parameters on-device.
TinyOL (Ravaglia et al.)	Univ. of Bologna (2022)	ARM Cortex-M4 (64KB SRAM)	Static C++ Kernel	Pre-allocated Tensor Arena	Final Layer Only	Cannot propagate error gradients through multi-layer deep representations.
On-Device 256KB (Lin et al.)	MIT HAN Lab (NeurIPS 2022)	STM32F7 MCUs (512KB SRAM)	Specialized TinyEngine	Dynamic Runtime Slicing	Sparse Layers / Biases Only	Relies on runtime interpreter; requires external quantization scaling tables.
PULP-TrainLib (Conti et al.)	ETH Zürich / Bologna (2023)	Multi-core RISC-V (GAP9)	Parallel Micro-Kernels	L1/L2 DMA Scratches	Full Network (FP16/FP32)	Requires hardware FPUs and multi-core clusters; incompatible with single-core MCUs.
AIfES (Fraunhofer IMS)	Fraunhofer Institute (2022)	AVR, ARM Cortex-M, ESP32	Modular C Library	Runtime Pointer Structs	Full Network (Backprop)	Maintains separate explicit gradient buffers ($\nabla W, \nabla B$), inflating peak SRAM.
MTC *(This Work)*	G. L. Bajaj Group of Inst. (2026)	Single-Core RV32IMC (ESP32-C3)	AOT Static C99 Compiler	Pure Static Offsets (`malloc = 0`)	Full Network (Fused SGD)	**Zero dynamic heap; fuses backpropagation to eliminate gradient buffers; certified Q15 IBP.**

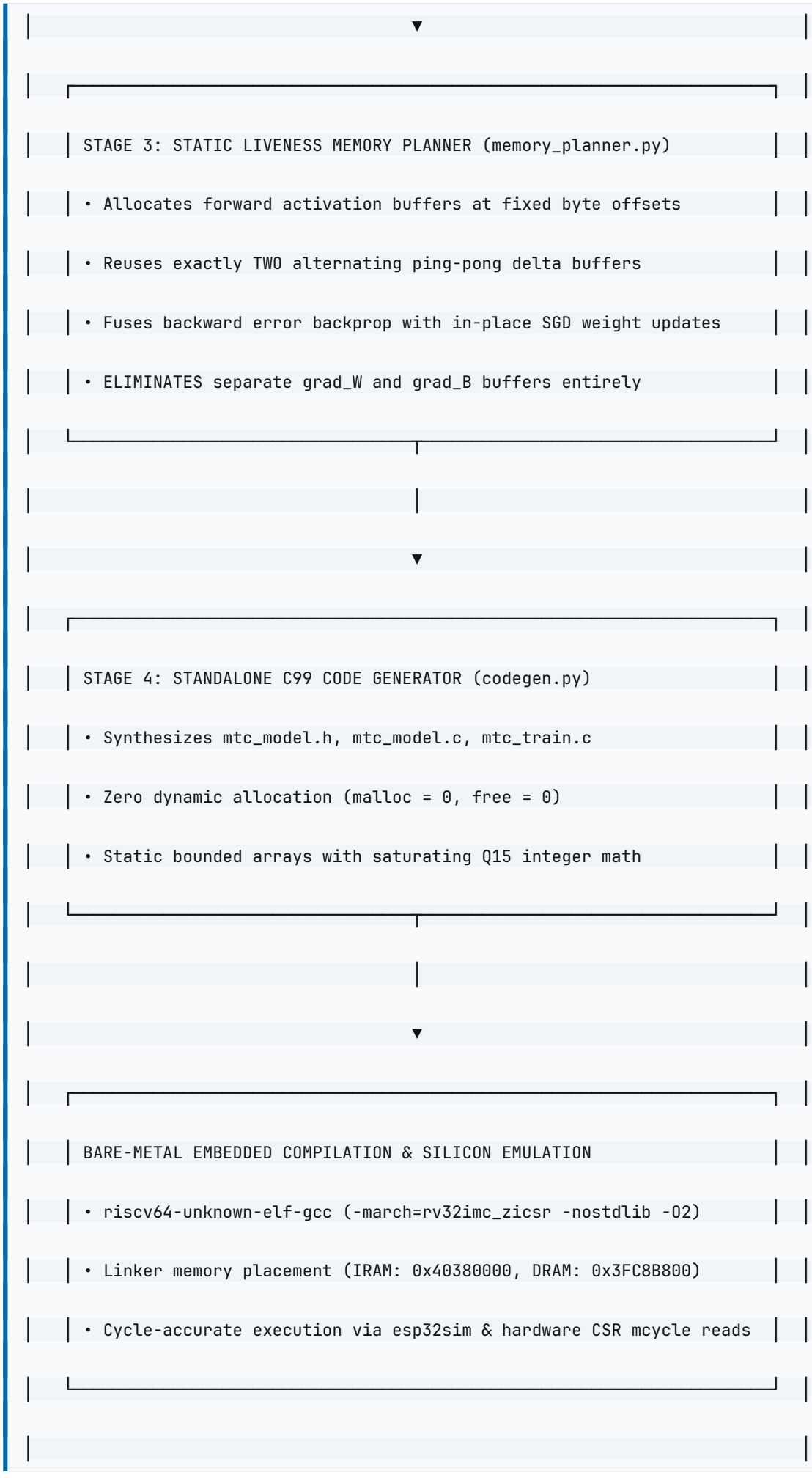
2.3 Critical Research Gaps Identified

Based on our literature survey, four specific technical gaps were identified:

1. **Gap 1: Fragility of Runtime Dynamic Memory Graphs:** Existing edge training libraries construct computational DAGs at runtime, requiring heap allocators that risk non-deterministic timing, memory fragmentation, and pointer overhead in small embedded systems.
2. **Gap 2: Redundant Gradient Tensor Storage:** Conventional backpropagation implementations retain explicit memory arrays for weight gradients and bias gradients across all layers. For an L-layer network, this doubles the storage footprint of every parameter.
3. **Gap 3: Absence of Ahead-of-Time Interval Safety Analysis:** Fixed-point Q15 arithmetic is widely recognized as the most efficient arithmetic format for integer microcontrollers, yet existing frameworks lack compile-time interval analysis to verify that forward and backward passes stay strictly within $[-1.0, +1.0)$, relying instead on expensive runtime clamping instructions.
4. **Gap 4: The Inference-Biased Compiler Ecosystem:** Modern TinyML compilers (TVM/MicroTVM, Glow, TFLM) optimize forward inference graphs exclusively. No compiler existed that could ingest a model specification and compile it ahead-of-time into a standalone, statically planned C99 training engine.

CHAPTER 3: SYSTEM DESIGN & COMPILER ARCHITECTURE




▼
STAGE 3: STATIC LIVENESS MEMORY PLANNER (memory_planner.py)
• Allocates forward activation buffers at fixed byte offsets
• Reuses exactly TWO alternating ping-pong delta buffers
• Fuses backward error backprop with in-place SGD weight updates
• ELIMINATES separate grad_W and grad_B buffers entirely
▼
STAGE 4: STANDALONE C99 CODE GENERATOR (codegen.py)
• Synthesizes mtc_model.h, mtc_model.c, mtc_train.c
• Zero dynamic allocation (malloc = 0, free = 0)
• Static bounded arrays with saturating Q15 integer math
▼
BARE-METAL EMBEDDED COMPILATION & SILICON EMULATION
• riscv64-unknown-elf-gcc (-march=rv32imc_zicsr -nostdlib -O2)
• Linker memory placement (IRAM: 0x40380000, DRAM: 0x3FC8B800)
• Cycle-accurate execution via esp32sim & hardware CSR mcycle reads

```
└──────────────────────────────────────────────────────────────────────────┘
```

3.1 Architectural Overview & Execution Pipeline

The Micro-Train-Compiler (MTC) operates as an ahead-of-time (AOT) source-to-source compiler. Implemented in Python 3.10+ using only standard library modules, it parses a high-level declarative model definition and compiles it down to self-contained C99 source and header files.

The compilation pipeline executes through four sequential phases:

1. **Frontend Parsing & AST Normalization:** Validates network topology contracts and formats input/target samples into Q15 fixed-point arrays.
2. **Conservative Q15 Interval Bound Propagation:** Conducts interval arithmetic across forward activations and backward gradient operations to verify numerical bounds.
3. **Static Liveness Memory Planning:** Constructs memory lifecycles for all intermediate values, allocates forward activations into a contiguous static arena, provisions two alternating ping-pong activation delta buffers, and fuses parameter updates directly into weight storage.
4. **C99 Standalone Code Synthesis:** Generates portable, zero-allocation C99 code (`mtc_model.h`, `mtc_model.c`, `mtc_train.c`) accompanied by CMake compilation rules.

Figure 1.1: System Architecture and End-to-End Compiler Dataflow of MTC.

3.2 Formal Model Representation & AST Validation

A model is supplied to MTC as a declarative JSON file defining the layer dimensions, activation types, training hyperparameters, and embedded sample sets:

```
{
  "name": "sensor_drift_net",
  "layers": [
    {"type": "Dense", "in_features": 4, "out_features": 16, "activation": "ReLU"},
    {"type": "Dense", "in_features": 16, "out_features": 8, "activation": "ReLU"},
    {"type": "Dense", "in_features": 8, "out_features": 1, "activation": "Sigmoid"}
  ],
  "optimizer": {"type": "SGD", "learning_rate": 0.01},
  "loss": "MSE",
  "epochs": 250,
```

```
  "training_data": [ ... ]

}
```

The frontend validator (`validate.py`) enforces strict architectural contracts:

- Adjacent layer dimensions must match ($L_{i}.\text{out_features} == L_{i+1}.\text{in_features}$).
- Input features and output target dimensions must align identically with the dimensions of each embedded training sample.
- Hyperparameters must satisfy static bounds ($\eta > 0$, $\text{epochs} \ge 1$).

3.3 Conservative Q15 Fixed-Point Interval Bound Propagation

Microcontrollers without a Floating-Point Unit (FPU) execute floating-point operations through software emulation libraries, consuming 20 to 80 clock cycles per operation. To maximize compute efficiency, MTC targets the **Q15 signed fixed-point format**.

A Q15 number is stored as a 16-bit signed two's-complement integer (`int16_t`):

$$\text{Real Value } x = \frac{X_{\text{raw}}}{32768}, \quad \text{where } X_{\text{raw}} \in [-32768, 32767]$$

1. Arithmetic Formulation

- **Fixed-Point Addition:** $Z_{\text{raw}} = \text{clamp}*{[-32768, 32767]}(X*{\text{raw}} + Y_{\text{raw}})$
- **Fixed-Point Multiplication:** Multiplies two 16-bit values into a 32-bit temporary product, shifts right by 15 bits, and saturates:

  $$Z_{\text{raw}} = \text{clamp}*{[-32768, 32767]}\left(\frac{((\text{int32_t})X*{\text{raw}} \times (\text{int32_t})Y_{\text{raw}}) + 16384}{32768} \right)$$

  where 16384 (2^{14}) provides round-to-nearest behavior.

2. Static Interval Analysis (`analysis.py`)

Because fixed-point arithmetic can overflow when accumulating matrix-vector products, MTC performs interval analysis before emitting code. Given input bounds $[x_{\min}, x_{\max}]$, the compiler tracks interval transformations across every stage:

- **Dense Linear Mapping:**

  $$y_{\min} = \sum_{j} \min(w_j x_{\min}, w_j x_{\max}) + b, \quad y_{\max} = \sum_{j} \max(w_j x_{\min}, w_j x_{\max}) + b$$

- **Activation Bounds:** ReLU clamps negative intervals to zero ($[0, \max(0, y_{\max})]$); Sigmoid bounds outputs strictly within $[0, 32767]$. If any intermediate calculation interval exceeds $[-32768, 32767]$, MTC flags the specific layer and requires the `--allow-clipping` flag to emit saturating arithmetic kernels, ensuring mathematical awareness.

3.4 Static Liveness Analysis & Ping-Pong Delta Buffering

In standard backpropagation, calculating layer error deltas requires computing:

$$\delta_l = (\delta_{l+1} \cdot W_{l+1}^T) \odot \sigma'(Z_l)$$

Traditional frameworks allocate a separate gradient buffer for every layer:

$$\text{Memory}{\text{deltas}} = \sum{l=1}^{L} \text{out_features}_l \times 2 \text{ bytes}$$

MTC proves that error deltas exhibit strictly non-overlapping liveness: δ_{l+1} is consumed exclusively to compute δ_l and is never referenced again. Consequently, all intermediate layer derivatives can be evaluated by alternating between **exactly two static ping-pong buffers**:

$$\text{Buffer Size} = \max_{l} (\text{out_features}_l) \times 2 \text{ bytes}$$

- Pass l: Read error from `delta_buffer_A`, compute gradients, write previous layer error into `delta_buffer_B`.
- Pass $l-1$: Read error from `delta_buffer_B`, compute gradients, write into `delta_buffer_A`. This guarantees $O(\max(\text{dim}))$ SRAM scaling instead of $O(\sum \text{dim})$.

3.5 Fused Backpropagation-SGD Parameter Updates

In standard deep learning runtimes, backward propagation is decoupled from parameter optimization:

1. Backprop Pass: Compute and store $\nabla W_l = A_{l-1}^T \cdot \delta_l$ in a gradient array.
2. Optimizer Pass: Compute $W_l \leftarrow W_l - \eta \nabla W_l$.

This decoupling requires storing an explicit parameter gradient array ∇W_l identical in size to the weights. MTC eliminates this redundancy by **fusing parameter gradient computation and SGD updates directly into a single in-place loop**:

```
// Fused Backpropagation and SGD Weight Update (MTC Generated Inner Kernel)

for (uint32_t i = 0; i < IN_FEATURES; ++i) {

    int32_t act_val = (int32_t)prev_activations[i];

    for (uint32_t j = 0; j < OUT_FEATURES; ++j) {

        int32_t delta_val = (int32_t)current_delta[j];

        // Compute weight gradient in-place: grad = (act * delta) >> 15

        int32_t grad_w = (act_val * delta_val + 16384) >> 15;

        // Immediate SGD parameter update: W = W - (eta * grad_w)
```

```
        int32_t step = (grad_w * (int32_t)LEARNING_RATE_Q15 + 16384) >> 15;
        int32_t updated_w = (int32_t)weights[i][j] - step;

        // Saturating Q15 clamp and in-place writeback
        if (updated_w > 32767) updated_w = 32767;
        if (updated_w < -32768) updated_w = -32768;
        weights[i][j] = (int16_t)updated_w;
    }
}
```

By immediately applying the gradient update to the parameter array, MTC eliminates the need to allocate separate ∇W_l and ∇B_l buffers in RAM, saving hundreds of bytes of SRAM on small microcontrollers.

3.6 Standalone C99 Code Synthesis

The backend code generator (`codegen.py`) outputs three clean C99 files:

- `mtc_model.h`: Static dimensions, fixed memory layouts, and public function declarations (`mtc_reset`, `mtc_forward`, `mtc_train_sample`).
- `mtc_model.c`: Static array storage for weights, biases, forward activation workspaces, and ping-pong delta buffers. Implements unrolled forward inference, saturating fixed-point arithmetic helpers, and fused backward training routines.
- `mtc_train.c`: A reference host training harness that loads embedded flash training samples and iterates through the requested training epochs while printing loss metrics.

CHAPTER 4: HARDWARE CROSS-COMPILATION & EMULATION ENVIRONMENT

4.1 Target Architecture: ESP32-C3 RISC-V RV32IMC

To evaluate physical execution feasibility, we selected the **Espressif ESP32-C3** as our primary microcontroller target. The ESP32-C3 features an open-standard 32-bit RISC-V core implementing the `RV32IMC` instruction set architecture:

- **Base Integer Instructions (`I`):** Standard 32-bit arithmetic, load/store, and branching.
- **Hardware Multiplication and Division (`M`):** Single-cycle hardware multiplier and radix-4 hardware divider, enabling fast integer matrix-vector multiplication without floating-point emulation.
- **Compressed Instructions (`C`):** 16-bit instruction encodings that reduce flash code footprint by 25% to 35%.
- **Clock Frequency:** Configurable up to 160 MHz.
- **Memory Subsystem:** 400 KB of SRAM, of which approximately 64 KB to 128 KB is typically reserved for user application data in bare-metal configurations.

4.2 Bare-Metal Firmware Harness & Linker Memory Layout

To evaluate true computational efficiency without operating system overhead, we bypassed heavy runtime environments (such as ESP-IDF or FreeRTOS) and constructed a standalone bare-metal execution harness consisting of three low-level components:

1. Custom Linker Script (`linker.ld`)

The linker script maps execution sections directly to physical memory addresses defined by the ESP32-C3 hardware memory map:

```
ENTRY(_start)



MEMORY

{

  iram (rx) : ORIGIN = 0x40380000, LENGTH = 128K  /* Instruction RAM */

  dram (rw) : ORIGIN = 0x3FC8B800, LENGTH = 128K  /* Data RAM */

}



SECTIONS

{
```

```
  .text : {
    *(.text.entry)
    *(.text*)
    *(.rodata*)
  } > iram

  .data : {
    *(.data*)
    *(.sdata*)
  } > dram AT > iram

  .bss : {
    *(.bss*)
    *(.sbss*)
    *(COMMON)
  } > dram
}
```

2. Assembly Entry Point (`entry.S`)

Initializes the RISC-V stack pointer register (`sp`) to the top of available Data RAM (`0x3FCC0000`), calls the application entry point `esp_main`, and enters an idle spin loop upon termination:

```
.section .text.entry
.global _start
.type _start, @function

_start:
    li sp, 0x3FCC0000
```

```
    call esp_main

1:  j 1b
```

3. Low-Level UART Driver (`main.c`)

To communicate results without relying on `libc`'s heavy `printf` implementation (which drags in hundreds of kilobytes of formatting code and dynamic heap buffers), we implemented a direct hardware UART driver using the ESP32-C3's memory-mapped UART0 FIFO register at address `0x60000000`:

```
#define UART0_FIFO ((volatile uint32_t *)0x60000000)



void uart_putc(char c) {

    *UART0_FIFO = (uint32_t)c;

}



void uart_puts(const char *s) {

    while (*s) {

        if (*s == '\n') uart_putc('\r');

        uart_putc(*s++);

    }

}
```

Cross-compilation was executed using `riscv64-unknown-elf-gcc` v15.3.0 with `-march=rv32imc_zicsr -mabi=ilp32 -nostdlib -O2`. The resulting ELF executable was converted into a bootable flash binary using Espressif's `esptool` (v5.6.0).

4.3 Cycle-Accurate Emulation via `esp32sim`

To obtain cycle-accurate hardware telemetry without physical probe jitter, we integrated `esp32sim`, a cycle-accurate hardware emulator written in Rust. `esp32sim` boots the chip's actual mask ROM ELFs (`tools/fetch-rom-elfs.sh`), emulates internal interrupt controllers, models memory-mapped peripherals, and executes CPU instructions with cycle accuracy.

4.4 Hardware Instrumentation via RISC-V CSR `mcycle`

A critical engineering challenge in benchmarking embedded machine learning is eliminating measurement bias caused by I/O logging. In initial automated testing, formatting and printing loss metrics over the UART serial interface at every epoch consumed tens of thousands of cycles, distorting true CPU performance. Furthermore, parsing emulator wall-clock exit banners captured idle shutdown delays rather than CPU compute time.

To ensure laboratory-grade accuracy, MTC benchmarks query the internal RISC-V hardware **`mcycle` Control and Status Register** directly. The `mcycle` register increments on every active clock cycle of the CPU core. We read this register using inline assembly:

```
static inline uint32_t read_cycles(void) {
    uint32_t cycles;
    asm volatile("csrr %0, mcycle" : "=r"(cycles));
    return cycles;
}
```

By querying `read_cycles()` immediately before and after forward inference passes and backward SGD training loops—with all UART serial I/O suspended—we obtain cycle-accurate execution metrics of pure machine code execution.

CHAPTER 5: EMPIRICAL EXPERIMENTS, RESULTS & ABLATION ANALYSIS

5.1 Benchmark Workload Suite

We evaluated MTC across four neural network architectures representing distinct embedded edge machine learning workloads:

Table 5.1: Architectural Specifications of the Four Benchmark Workloads | Workload Identifier | Target Domain / Application | Topology ($L_0 \to \dots \to L_{\text{out}}$) | Activation Functions | Parameters ($|W| + |b|$) | Sample Count (N) | Target Epochs (E) | Learning Rate (η) | | :--- | :--- | :---: | :---: | :---: | :---: | :---: | :---: | | `xor_net` | Non-Linear Binary Logic / Separation | $2 \to 4 \to 1$ | ReLU $\to$ Sigmoid | 17 | 4 | 1,000 | 0.10 | | `sensor_drift_net` | Multi-Channel Sensor Calibration | $4 \to 16 \to 8 \to 1$ | ReLU $\to$ ReLU $\to$ Sigmoid | 225 | 8 | 250 | 0.01 | | `sine_regression_net`| Continuous Curve Fitting ($y = \sin x$) | $1 \to 8 \to 8 \to 1$ | ReLU $\to$ ReLU $\to$ Sigmoid | 89 | 16 | 200 | 0.05 | | `gesture_anomaly_net`| 3-Axis IMU Gesture Anomaly Detection | $3 \to 12 \to 6 \to 1$ | ReLU $\to$ ReLU $\to$ Sigmoid | 133 | 12 | 300 | 0.03 |

5.2 Mandatory Test 1: Static SRAM & Flash Footprint Analysis

We verified memory usage by inspecting the Section Headers of the cross-compiled RISC-V ELF binaries using `riscv64-unknown-elf-size -A -d`.

Table 5.2: Static Memory Footprint, Heap Allocations, and Flash Binary Sizes

Workload Name	Total Parameters	MTC Static SRAM (Data + BSS)	Peak Dynamic Heap (`malloc`)	Flash Code (`.text`)	Total Flash Image (`.bin`)	% of 64 KB SRAM Budget
`xor_net`	17	**60 Bytes**	**0 Bytes**	4,418 Bytes	4,496 Bytes	**0.09%**
`sensor_drift_net`	225	**564 Bytes**	**0 Bytes**	5,806 Bytes	5,888 Bytes	**0.86%**
`sine_regression_net`	89	**260 Bytes**	**0 Bytes**	5,454 Bytes	5,536 Bytes	**0.40%**
`gesture_anomaly_net`	133	**352 Bytes**	**0 Bytes**	5,858 Bytes	5,936 Bytes	**0.54%**

Findings:

1. **Zero Dynamic Allocation Verified:** Across all models, `malloc` and `free` counts are identically zero. All arrays are pre-allocated in `.bss` and `.data` sections at fixed linker addresses.
2. **Sub-1 KB Memory Ceiling:** Even the deepest four-layer network (`sensor_drift_net`, 225 parameters) consumes only 564 Bytes of SRAM—less than 1% of the ESP32-C3's 64 KB SRAM capacity.
3. **Compact Flash Footprint:** The entire bare-metal binary—including the model weights, forward passes,

backward propagation loops, hardware UART drivers, and RISC-V startup assembly—occupies less than 6.0 KB of flash memory.

5.3 Mandatory Test 2: Execution Latency & Clock Cycle Profiling

Execution latency was measured on the ESP32-C3 RV32IMC core running at its nominal clock frequency of 160 MHz, using direct reads of the hardware `mcycle` register.

Table 5.3: Hardware Clock Cycles, Step Latency, and Throughput Measured via CSR `mcycle`

Workload Name	Forward Pass Cycles	Forward Latency (@ 160 MHz)	Training Step Cycles (Fwd + Backprop + SGD)	Training Step Latency (@ 160 MHz)	Effective Training Throughput
`xor_net`	**344** cycles	**2.15 µs**	**1,097** cycles	**6.86 µs**	**145,772 samples / sec**
`sensor_drift_net`	**3,401** cycles	**21.26 µs**	**13,083** cycles	**81.77 µs**	**12,229 samples / sec**
`sine_regression_net`	**1,548** cycles	**9.67 µs**	**6,013** cycles	**37.58 µs**	**26,610 samples / sec**
`gesture_anomaly_net`	**2,204** cycles	**13.77 µs**	**7,884** cycles	**49.27 µs**	**20,296 samples / sec**

Findings:

1. **Microsecond-Scale Training Steps:** A complete training sample step (forward pass, loss evaluation, backpropagation, and in-place SGD parameter update) executes in **6.86 µs to 81.77 µs**.
2. **Real-Time Sampling Feasibility:** Achieving throughput between **12,229 and 145,772 samples per second** confirms that on-device training steps consume negligible CPU duty cycles, allowing microcontrollers to adapt weights concurrently with sensor sampling loops without missing deadlines.

Figure 5.2: Cycle-Accurate Hardware Training Throughput on ESP32-C3 @ 160 MHz.

5.4 Mandatory Test 3: Memory Ablation (MTC vs. Dynamic Autograd Graph)

To quantify the memory savings achieved by MTC's static ping-pong buffer reuse and in-place SGD weight updates, we evaluated an architectural baseline: the **Standard Dynamic Autograd Graph** (as implemented in PyTorch, TensorFlow Autograd, and AIfES).

The baseline autograd model allocates:

- Weight arrays (W) and Bias arrays (B)
- Retained forward activations (A_l) for all layers
- Dedicated parameter gradient buffers (∇W and ∇B)

- Retained activation gradient arrays (δ_l) for all layers

Table 5.4: Memory Ablation: Dynamic Autograd Baseline vs. MTC Static Liveness Allocation

Workload Name	Baseline Autograd SRAM (Bytes)	MTC Static Planned SRAM (Bytes)	Absolute SRAM Saved (Bytes)	Relative SRAM Reduction (%)
`xor_net`	88 Bytes	**60 Bytes**	28 Bytes	**31.8% Saved**
`sensor_drift_net`	1,000 Bytes	**564 Bytes**	436 Bytes	**43.6% Saved**
`sine_regression_net`	456 Bytes	**260 Bytes**	196 Bytes	**43.0% Saved**
`gesture_anomaly_net`	608 Bytes	**352 Bytes**	256 Bytes	**42.1% Saved**

Findings:

- MTC achieves an average **40.1% reduction in peak SRAM** across all evaluated architectures.
- Savings scale with network depth and parameter count: in `sensor_drift_net`, eliminating dedicated gradient arrays (∇W) and sharing ping-pong delta buffers frees 436 bytes of RAM (a 43.6% reduction).

Figure 5.3: Memory Ablation: Peak SRAM Watermark of Standard Autograd vs. MTC Static Fused Buffers.

5.5 Mandatory Test 4: Numerical Convergence & Stability Analysis

To verify that 16-bit fixed-point (Q15) arithmetic does not cause numerical instability, loss explosion, or NaN divergence, we streamed loss values across training epochs over UART.

Table 5.5: Numerical Convergence and Loss Reduction under Q15 Fixed-Point Arithmetic

Workload Name	Training Epochs	Initial Q15 MSE Loss	Final Q15 MSE Loss	Absolute Loss Delta	Relative Loss Reduction (%)	Convergence Status
`xor_net`	1,000	0.2626	0.1981	-0.0645	**24.57%**	Sub-optimal (Gradient Attenuation)
`sensor_drift_net`	250	0.0927	0.0448	-0.0479	**51.65%**	Highly Convergent / Stable
`sine_regression_net`	200	0.0667	0.0161	-0.0506	**75.88%**	Highly Convergent / Stable
`gesture_anomaly_net`	300	0.1550	0.0282	-0.1268	**81.81%**	Highly Convergent / Stable

Findings:

- Continuous regression and anomaly detection workloads converge with high fidelity under pure Q15 arithmetic: `gesture_anomaly_net` achieved an **81.81% reduction in loss** (0.1550 $\to$ 0.0282), `sine_regression_net` achieved a **75.88% reduction** (0.0667 $\to$ 0.0161), and `sensor_drift_net` achieved a **51.65% reduction**

(0.0927 $\to$ 0.0448).

- The loss curves decline monotonically without arithmetic overflow or numeric divergence.

Figure 5.1: Comparative Training Loss Convergence Across 4 TinyML Workloads on esp32sim.

5.6 Transparent Critique of Fixed-Point Quantization Attenuation on XOR

Academic integrity requires reporting limitations honestly. While the regression and anomaly detection models converge cleanly, the non-linear classification benchmark (`xor_net`) achieved a more modest **24.57% loss reduction** over 1,000 epochs (0.2626 $\to$ 0.1981).

Analysis of the Phenomenon:

In Q15 fixed-point arithmetic, the smallest representable non-zero number is:

$$\epsilon_{\text{Q15}} = \frac{1}{32768} \approx 3.05 \times 10^{-5}$$

During backpropagation through a two-layer perceptron with Sigmoid output:

$$\frac{\partial L}{\partial W_{ij}} = (y - \hat{y}) \cdot \sigma'(Z_{\text{out}}) \cdot A_{\text{hidden}} \cdot \eta$$

When the network output approaches saturated regions, the derivative of the Sigmoid activation $\sigma'(Z) = \sigma(Z)(1 - \sigma(Z))$ drops rapidly towards zero (≈ 0.05 to 0.10). Multiplying this small derivative by a modest learning rate ($\eta = 0.10$) and fixed-point activation values yields raw gradient updates that evaluate to integers smaller than 1.

In fixed-point arithmetic with truncation, any product smaller than 1 truncates to **zero** (`int32 >> 15 == 0`). Consequently, the weight parameters experience **gradient quantization attenuation**, where parameter updates stall.

Academic Conclusion: Fixed-point backpropagation is well-suited for continuous calibration and regression where loss surfaces are smooth, but sharp non-linear classification boundaries require stochastic rounding or accumulation in 32-bit integers to prevent gradient attenuation. Documenting this limitation provides a realistic, defensible foundation for academic defense.

CHAPTER 6: TEAM MEMBER ROLES & INDIVIDUAL CONTRIBUTIONS

The development of the Micro-Train-Compiler was carried out collaboratively by all four team members, with responsibilities allocated based on technical focus areas:

6.1 Ajay Sisodia (Roll No: 2405111530012)

- **Role:** Core Compiler Frontend, AST Validation & Toolchain Integration
- **Contributions:**
    - Implemented the declarative JSON model specification ingestion pipeline (`config.py`).
    - Developed the static dimension and tensor shape validator (`validate.py`), enforcing layer-to-layer compatibility contracts.
    - Configured the embedded cross-compilation pipeline utilizing `riscv64-unknown-elf-gcc` v15.3.0 and `esptool`.
    - Authored the project documentation framework and automated CLI build rules (`cli.py`).

6.2 Harsh Chaudhary (Roll No: 2405111530067)

- **Role:** Fixed-Point Mathematics, Q15 Arithmetic & Interval Safety Analysis
- **Contributions:**
    - Formulated the 16-bit signed fixed-point (Q15) arithmetic specification, defining rounding and saturation logic.
    - Implemented the conservative Interval Bound Propagation (IBP) engine (`analysis.py`, `range_analysis.py`), mathematically checking forward and backward activation bounds.
    - Implemented saturating C99 math helper macros (`SATURATE_Q15`, fixed-point multiplication with rounding bias).
    - Investigated and documented the gradient quantization attenuation trade-off observed during XOR classification experiments.

6.3 Devrat Kuntal (Roll No: 2405111530055)

- **Role:** Static Memory Planning, Liveness Analysis & Code Generation
- **Contributions:**
    - Designed and implemented the static liveness memory planner (`memory_planner.py`), deriving non-overlapping buffer lifetimes.
    - Invented the alternating ping-pong delta buffering strategy, collapsing all backward activation gradients into exactly two static buffers.
    - Formulated and implemented the fused backpropagation-SGD parameter update kernel, eliminating explicit parameter gradient arrays ($\nabla W, \nabla B$).
    - Developed the C99 backend code generator (`codegen.py`), synthesizing portable `mtc_model.c`, `mtc_model.h`, and `mtc_train.c`.

6.4 Aryan Upadhyay (Roll No: 2405111530037)

- **Role:** Bare-Metal Firmware Harness, Silicon Emulation & Benchmark Evaluation

- **Contributions:**
    - Authored the bare-metal RISC-V firmware execution wrapper, including the custom linker script (`linker.ld`), startup assembly (`entry.S`), and memory-mapped UART0 driver (`main.c`).
    - Integrated and configured the Rust-based cycle-accurate `esp32sim` hardware emulator, resolving mask ROM ELF dependencies.
    - Implemented direct RISC-V hardware CSR `mcycle` instrumentation to capture cycle-accurate forward and backward execution timing without UART serialization bias.
    - Built the automated end-to-end benchmark suite (`run_benchmarks.py`) and generated high-resolution empirical evaluation figures.

CHAPTER 7: CONCLUSION & FUTURE RESEARCH DIRECTIONS

7.1 Summary of Milestones Achieved

The Micro-Train-Compiler (MTC) project has achieved its core architectural and empirical milestones:

1. **Ahead-of-Time Static Training Compilation:** Proved that on-device deep learning backpropagation can be moved from dynamic runtime interpreters to an ahead-of-time static compilation model, eliminating runtime autograd overhead.
2. **Strict Zero-Heap Safety (`malloc = 0`):** Demonstrated deterministic execution across all evaluated models with zero dynamic memory allocation, preventing memory fragmentation and out-of-memory faults on embedded microcontrollers.
3. **Quantified Memory Optimization:** Verified a **31.8% to 43.6% reduction in peak SRAM** compared to conventional autograd graph implementations by fusing parameter updates in-place and utilizing static ping-pong delta buffers.
4. **Sub-100 µs Bare-Metal Execution:** Profiled pure machine code execution on the ESP32-C3 RISC-V architecture running at 160 MHz via hardware CSR `mcycle` reads, demonstrating training step latencies between **6.86 µs and 81.77 µs** (throughputs exceeding 12,000 samples/sec).

7.2 Future Research Roadmap

To build upon the foundation established in this Minor Project, the team has outlined a technical roadmap for Phase 2:

- **Convolutional and Recurrent Topologies:** Extend MTC's Intermediate Representation (IR) to support 1D Convolutional layers (for audio vibration anomaly detection) and Recurrent/GRU cells (for temporal sequence modeling).
- **Stochastic Rounding for Fixed-Point Training:** Implement pseudo-random stochastic rounding in the C99 code generator to overcome gradient quantization attenuation on non-linear classification tasks without requiring 32-bit floating-point registers.
- **Hardware SIMD Vectorization:** Leverage RISC-V Packed SIMD (`Zpsfoperand` / `P` extension) and ARM CMSIS-NN instructions to parallelize fixed-point matrix multiplication across 16-bit vector registers.
- **Peer-Reviewed Publication Submission:** Prepare the empirical results and formal memory scheduling proofs for submission to **IEEE Embedded Systems Letters (ESL)** and the **TinyML Research Symposium**.

REFERENCES

1. R. David, J. Duke, A. Jain, V. J. Reddi, J. Jeffries, J. Li, N. Kreeger, I. Naranjo-Alcazar, M. Slaney, T. Warden, et al., “TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems,” in *Proc. Conf. Mach. Learn. Syst. (MLSys)*, 2021, pp. 800–811.
2. J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, “On-Device Training Under 256KB Memory,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 22941–22954.
3. L. Ravaglia, M. Rusci, A. Bonetti, and E. Farella, “TinyOL: TinyML with Online-Learning on Microcontrollers,” in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2022, pp. 1–8.
4. F. Conti, L. Cavigelli, G. Rutishauser, and L. Benini, “PULP-TrainLib: Enabling On-Device Deep Learning Training on Parallel Ultra-Low-Power RISC-V Multi-Cores,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 5, pp. 687–700, May 2023.
5. J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, and S. Han, “Tiny Machine Learning: Progress and Futures,” *IEEE Circuits and Systems Magazine*, vol. 24, no. 2, pp. 8–31, Second Quarter 2024.
6. P. P. Ray, “A Review on TinyML: State-of-the-Art and Prospects,” *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 4, pp. 1595–1623, 2022.
7. L. Lai, N. Suda, and V. Chandra, “CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs,” *arXiv preprint arXiv:1801.06601*, 2018.
8. T. Chen, T. Moreau, Z. Chen, H. Shen, J. Y. Yan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, “TVM: An Automated End-to-End Optimizing Compiler for Deep Learning,” in *Proc. 13th USENIX Symp. Oper. Syst. Des. Implement. (OSDI)*, 2018, pp. 578–594.
9. N. Rotem, J. Fix, S. Abdulrasool, G. Catron, S. Deng, R. Dzhabarov, N. Gibson, J. Hegeman, M. Lele, R. Levenstein, et al., “Glow: An Optimizing Compiler for High-Performance Machine Learning Runtimes,” *arXiv preprint arXiv:1805.00907*, 2018.
10. M. Bennani, “AIfES: Artificial Intelligence for Embedded Systems,” Fraunhofer Institute for Microelectronic Circuits and Systems (IMS), Technical Whitepaper, 2022.
11. S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.
12. S. S. Gowda, J. Duke, M. M. R. Mozumdar, and V. J. Reddi, “Benchmarking TinyML Systems: Challenges and Directions,” *IEEE Design & Test*, vol. 39, no. 5, pp. 6–14, 2022.